유경동 자기소개서

Unity·C# 게임 클라이언트 · 씨플레이 CatJump 재직

1. 라이브 이벤트의 보상·저장·광고 복귀 경로

2025년 7월부터 씨플레이에서 모바일 게임 CatJump의 Unity·C# 클라이언트를 개발하고 있습니다. CatJump는 Google Play 다운로드 500만 이상, 약 70만 MAU 규모로 운영 중인 게임입니다. 저는 약 2~3주 주기 업데이트에 맞춰 기간 한정 이벤트, 보상·스킨 같은 메타 기능, 10인 실시간 PvP를 맡아 왔습니다.

메인 화면에서 광고를 보고 돌아오면 티켓 보상이 저장되지 않는 경로가 있었습니다. 광고 결과 콜백, 보상 지급, 저장, UI·알림 갱신이 이 순서로 이어지도록 경로를 맞췄고, 이어하기 광고의 스킵·실패 예외도 처리했습니다. 신규 미니게임은 매번 독립 시스템으로 만들지 않고 진입·결과·보상·저장·기간 종료로 이어지는 공통 이벤트 흐름에 연결했습니다. 보상 상자에서는 표시 계층이 확정된 결과만 받도록 지급·저장과 팝업 표시를 나눴고, 추첨 전에 획득할 수 없는 후보를 제외했습니다.

2. 10인 PvP의 호스트 변경 복구와 순번 카운트다운

CatJump의 10인 실시간 점프 레이스를 Photon PUN2로 구현했습니다. 경기 흐름 설계부터 출시 후 오류 수정까지 제가 맡았습니다. 전용 서버 없이 마스터 클라이언트가 진행 권한을 갖는 구조이고, 플레이 환경은 모바일 네트워크입니다. 이 구조에서는 마스터가 경기 중에 나가면 준비·카운트다운·결과 처리가 멈추거나 클라이언트마다 다르게 진행될 수 있었습니다. 이벤트를 받는 시점도 클라이언트마다 달라 늦게 도착한 카운트다운이 현재 상태를 덮어쓸 수 있었고, 경기 중 이탈로 인원이 줄면 레이스의 경쟁감이 떨어졌습니다.

먼저 경기를 매칭 탐색, 인원 확정, 준비·카운트다운, 플레이, 결과 단계로 나누고, 진행 단계의 복구 기준은 룸 속성에 두었습니다. 새 마스터가 이 값을 읽어 경기를 재개하면 각 클라이언트의 단계 표시도 같은 값을 따릅니다. 카운트다운 이벤트에는 순번을 붙여, 이미 지난 순번이 늦게 도착하면 버립니다. 경기 중 이탈한 유저의 자리는 AI로 채워 남은 참가자의 레이스가 끊기지 않습니다. 출시 후에는 카운트다운 불일치, 룸 진입 크래시, 로비 복귀 버그를 수정했습니다.

3. 이벤트별 하드코딩 대사를 공용 시스템으로

CatJump의 기간 한정 이벤트에는 캐릭터 대사와 연출이 자주 들어갑니다. 공용 시스템이 생기기 전에는 이벤트마다 대사와 연출을 코드에 직접 작성했고, 새 이벤트를 만들 때마다 대사 UI 연결, 종료 처리, 예외 처리를 다시 구현해야 했습니다.

공용 대사 시스템은 설계와 구현을 처음부터 맡았습니다. 대사 내용과 순서는 시트에서 만든 데이터 자산으로 관리하고, 대사 진행 흐름과 UI 표시는 서로 다른 책임으로 나눴습니다. 태그를 깨지 않는 리치 텍스트 타이핑, 뒤로가기 닫기, 대사가 끝난 뒤 다음 동작을 넘기는 완료 콜백은 공용 시스템에 한 번만 구현했습니다. 이미 하드코딩으로 만든 기존 이벤트와 함께 동작해야 했고, 현지화 리소스와도 맞물려야 했습니다. 이후 다른 개발자가 만든 이벤트에서도 이 시스템을 썼습니다.

4. 팀 프로젝트 통합과 예외 경로 정리

CatJump에서는 이탈, 재진입, 광고 복귀, 중복 수령, 기간 종료 같은 예외 경로를 재현 가능한 케이스로 정리해 QA 확인 항목으로 넘겼습니다. 개발 환경에서 재현되지 않는 크래시는 Crashlytics 리포트로 발생 단계와 데이터 조건을 좁혀 확인 항목을 만들었습니다. 레벨 에디터 업데이트와 종료 이벤트 자산 정리, 현지화 리소스 반영도 맡았습니다.

DARK SHIELD에서는 3인 팀장을 맡았습니다. 따로 만든 매니저들을 합치면서 Awake 실행 순서에 따라 적이 플레이어를 찾지 못하거나 상점 상호작용이 연결되지 않는 문제가 생겼습니다. StageManager의 개별 싱글톤을 없애고, GameManager가 명시적으로 Init을 호출하는 순서로 정리했습니다. 플레이어 FSM·무기·스킬과 적 AI·풀링은 팀원 담당이었고, 저는 스테이지 흐름과 매니저 통합을 맡았습니다. Jumac에서도 팀장으로 기획자 작업용 씬을 분리하고 기획 UI·맵을 개발 씬에 통합했습니다.

1인 C++ 콘솔 게임 MAZEGAME에서는 정면 벽 거리와 좌우 통로 길이로 1인칭 문자 화면을 합성했습니다. 렌더러가 늘 배열 위쪽을 전방으로 읽기 때문에, 방향 전환은 보드를 회전한 뒤 플레이어 좌표를 다시 찾는 방식으로 연결했습니다.

5. VR 리듬 게임의 음악·노트 시간 기준

NT는 9인 팀(개발 5·기획 4, 저장소 README 기준)이 만든 VR 리듬 게임이고, 저는 개발팀장으로 노트 생성·이동과 BGM·타격음 재생을 맡았습니다. 초기 노트는 고정 스텝으로 움직여서 생성이 늦어진 만큼 판정선 도착도 늦어졌습니다. BGM은 AudioSource가, 노트는 MonoBehaviour가 움직여 기준 시계도 서로 달랐습니다.

시작 시각 하나를 오디오 엔진 시계(dspTime)로 정하고, 그 값으로 BGM을 PlayScheduled로 예약했습니다. 같은 값을 모든 노트 생성기에도 넘겨, 노트 목표 시각은 시작 시각에 곡 내 시각을 더해 계산하고 위치는 매 프레임 DSP 진행률로 다시 구하게 했습니다. 노트가 삭제될 때 다음 키음을 고르던 구조는 레일별 (시각, 클립) 시간표로 바꿔, 노트가 없는 빈 타격에도 낼 소리가 정해지게 했습니다. 판정·Photon 2인 멀티·곡 에디터는 팀원이 맡았습니다.

코드 탐색·작업 분해·체크리스트 정리에는 Codex CLI와 Claude Code를 보조로 쓰고, 결과는 직접 확인합니다. 서울·경기·부산 모두 근무할 수 있고 수도권으로 이주할 수 있습니다.